INTERVIEW PREPARATION GUIDE · OCTOBER 2026

School Management Portal

A source-accurate project walkthrough, technical reference, and practice question bank for explaining the application in an interview.

Explain the project with confidence

Understand the user workflows, Flask architecture, database relationships, validation, authorization, result calculations, and the tradeoffs you would address before production.

Python · Flask · SQLAlchemy · SQLite · Jinja · Bootstrap

What's inside

Project story

Elevator pitch, problem, scope, users, and feature workflows.

Technical walkthrough

Routes, templates, models, constraints, validation, sessions, and calculations.

Interview question bank

Practical questions with concise answers grounded in the current source.

Honest next steps

Security, testing, migrations, deployment, scaling, and limitations to discuss.

Use this honestly: Adapt first-person answer examples to the work you actually did. The guide explains the current code; it does not claim that unimplemented improvements or tests already exist.

01 · THE PROJECT STORY

Pitch, purpose, scope

30-second introduction

> "This is a server-rendered school management web app built with Python, Flask, Flask-SQLAlchemy, and SQLite. It supports administrator, teacher, and student roles. Staff manage student profiles, subjects, examination marks, and results, while students can view only their own profile and academic results. Flask-Login handles authentication, WTForms validates submissions and provides CSRF protection, and Jinja templates render a Bootstrap-based responsive interface. It is a learning/demo application, and I can explain the security and deployment work needed before real school use."

Problem it addresses

The application gives a school a single place to maintain student/contact records, configure subjects, enter assessment scores, and review academic summaries. It replaces a set of disconnected manual screens with role-specific web workflows.

Users and primary goals

User	Goal	Examples
Administrator	Configure and oversee school data	Create student accounts, add subjects, enter/review marks, manage account activation.
Teacher	Work with academic data	Browse students, view profiles/subjects, enter marks, filter results.
Student	See personal academic information	View own dashboard, profile, marks, and results.

Feature scope

- Login/logout and hashed passwords.
- Role-aware dashboards and protected routes.
- Student directory, profile, search, and grade filter.
- Subject catalogue and subject maximum marks.
- Exam marks create/update, result totals, percentages, grades, and filters.
- User account listing and activation/deactivation for administrators.

Not included: attendance, fee collection, timetables, parent portal, password reset, PDF report-card export, or audit history.

02 · HOW IT WORKS

Architecture and request flow

This is a traditional server-rendered Flask web application. The browser submits a request; Flask handles route logic and ORM operations; Jinja renders HTML; the browser receives the page. There is no separate JavaScript SPA or REST API layer in the current implementation.

```
Browser
  | HTTP request / HTML form
  v
Flask route (app.py)
  | auth + role checks; WTForms validation
  | SQLAlchemy queries / writes
  v
SQLite database
  |
Flask renders Jinja template + shared base.html
  |
  v
Browser displays HTML + Bootstrap/custom CSS
```

Example: recording marks

1. Administrator or teacher opens `/results/marks`.
2. The route loads student and subject choices from the database.
3. The browser submits the WTForms form, including a CSRF token.
4. WTForms validates types/required fields; the route fetches the chosen student and subject.
5. The route rejects marks above the subject maximum; otherwise it finds the unique student/subject/exam/year record.
6. It updates the score or inserts a new record, commits, flashes a confirmation, and redirects to results.

Shared layout

`base.html` owns the navigation, user badge, flash messages, page heading, Bootstrap assets, and common footer. Child templates extend it and fill named Jinja blocks. This reduces duplicated HTML and allows page content to vary by role.

Important implementation boundary

Hiding a button in a template is only a user-interface choice. The routes independently enforce access with `@login_required` or `@roles_required`; student-specific routes also check profile ownership.

03 · CODE MAP

Files, routes, and framework responsibilities

File/group	Responsibility
`app.py`	Flask setup, config, ORM models, WTForms, helper functions, routes, error handlers, demo seeding, startup.
`templates/base.html`	Shared authenticated app shell and anonymous login shell.
`templates/*.html`	Login, staff/student dashboards, student list/forms/profile, subjects, marks, results, users, and error pages.
`static/css/style.css`	Custom visual system and responsive layout, supplementing Bootstrap.
`requirements.txt`	Flask, Flask-SQLAlchemy, Flask-Login, Flask-WTF, and email-validator dependency ranges.
`README.md`	Setup, demo credentials, workflows, and project limitations.

Route map

Route	Access	Purpose
`/`	Public	Redirect to dashboard if signed in; otherwise login.
`/login`, `/logout`	Public / signed-in	Authenticate and end session.
`/dashboard`	Signed in	Render staff dashboard or personal student dashboard.
`/students`	Admin, teacher	Search and filter student directory.
`/students/new`	Admin	Create linked student profile and login.
`/students/<id>`	Signed in, ownership scoped for students	Profile plus examination history.
`/subjects`	Admin, teacher; admin writes	View subject catalogue; administrators add subjects.
`/results`	Signed in	Role-scoped result register and staff filters.
`/results/marks`	Admin, teacher	Create or update marks.
`/users`, `/users/<id>/toggle`	Admin	List users and toggle other accounts.

04 · DATA AND BUSINESS RULES

Database model and relationships

Model	Important fields	Relationship / constraint
User	id, username, password_hash, role, display_name, active	Unique username. Student profile is optional one-to-one.
Student	admission_no, full_name, class_name, section, contact fields, user_id	Unique admission number; optional unique FK to User. One student has many marks.
Subject	name, code, max_marks	Unique subject name and code.
Mark	student_id, subject_id, exam_name, academic_year, marks_obtained, updated_at	Foreign keys to Student/Subject. Unique composite key across student, subject, exam, and year.

Why the composite uniqueness constraint?

The domain rule is one score for a student's subject in a given exam/year. A database constraint is the final guard against duplicates, even if application logic changes or concurrent requests race. The route first searches that same combination, then updates it or inserts a new row.

Student and login association

`Student.user_id` is optional and unique. A student record can therefore exist without a portal login, while a linked student account resolves to one profile. Student-specific pages use the association to scope visible records.

Delete behavior

The Student→Mark ORM relationship uses `cascade="all, delete-orphan"`. That expresses that a student's marks belong to the student record at the ORM level. Production deletion policy should still be deliberate and tested; an audit-sensitive school system may prefer soft deletion/history rather than permanent deletion.

Result calculations

```
obtained = sum(each recorded marks_obtained)
maximum  = sum(each corresponding subject.max_marks)
percent  = obtained / maximum * 100, if maximum is nonzero
```

The calculation is weighted by maximum points, not an average of subject percentages. The grade mapping is A+ ≥90, A ≥80, B+ ≥70, B ≥60, C ≥50, D ≥40, else F. With no marks, the helper returns a dash grade.

Data choices to know: `Student.class_name` and `date_of_birth` are strings, and academic year/exam choices are hard-coded in form/template code. A mature system would use stronger types and configurable academic-year/exam entities.

Sessions, authorization, validation

Authentication lifecycle

1. User submits username/password to `/login`.
2. WTForms validates presence and username length.
3. App loads user, checks `active`, then uses Werkzeug `check_password_hash`.
4. On success, Flask-Login's `login_user` establishes authentication and redirects to dashboard.
5. On each protected request, Flask-Login provides `current_user`; the user loader retrieves the account by ID.

Authorization

`roles_required(*roles)` wraps a route with login protection and role membership. If a signed-in user's role is not allowed, it aborts with 403. Ownership checks are separate: for example, the student profile route rejects a student whose `user_id` does not match `current_user.id`.

Form validation examples

Form	Examples of rules
Login	Username/password required; username length capped.
Student	Name/admission/user fields required; lengths bounded; optional email validated; password minimum length if supplied; class/section/gender use selects.
Subject	Name/code required; maximum marks integer from 1–1000.
Mark	Student/subject IDs coerced to integers; score integer from 0–1000; exam and year are chosen options; route separately checks subject-specific maximum.

CSRF and rendering

Forms derive from `FlaskForm`; templates call `form.hidden_tag()`, which emits the hidden CSRF token. Jinja autoescaping helps escape ordinary interpolated text. CSRF protection does not replace authentication, authorization, validation, or HTTPS.

Password hashing

`generate_password_hash` creates the stored password hash and `check_password_hash` verifies a login. The source does not store plaintext passwords for normal account records. Demo credentials remain a separate risk because their values are known and seeded.

06 · INTERVIEW PRACTICE

Core questions: product and architecture

1. Tell me about this project.

"It is a Flask/SQLite school portal for student records and academic results. Admins and teachers manage appropriate school information, while student users see their own profile and results. It uses SQLAlchemy for persistence, Flask-Login for sessions, Flask-WTF for forms/CSRF, and Jinja/Bootstrap for server-rendered pages. It is a demo, so I would harden security, testing, migrations, and deployment before real use."

2. What problem were you solving?

"The goal was to bring student records, subject setup, examination marks, and results together in a simple role-aware interface rather than treating each task as a separate manual process."

3. Who are the users?

"There are administrator, teacher, and student roles. The role determines which routes and actions are available; students are also scoped to their own linked student profile."

4. Why Flask instead of a larger framework?

"Flask kept a small learning project lightweight and let me choose focused extensions for login, ORM, and forms. If the app grew, I would split it into blueprints and add clearer service/configuration boundaries."

5. Is this a REST API or a single-page application?

"Neither. It is a traditional server-rendered application: routes query or update the database and return Jinja-rendered HTML. A separate API and frontend could be added if clients or product needs required them."

6. Walk me through a request.

"Flask dispatches the URL to a route. Authentication and role checks run first, then the route validates form data, performs SQLAlchemy queries or writes, and passes data into a Jinja template. The rendered HTML is returned as the response."

7. What does Jinja template inheritance achieve?

"A shared base template owns navigation and page chrome. Child templates provide their title, heading, and page content, so shared UI stays consistent without copying it across each page."

8. What are the main components in app.py?

"Application configuration/extensions, SQLAlchemy models, WTForms classes, login/role helpers, grade/result helpers, route functions, error handlers, demo seed logic, and app startup. It is compact but would be divided into modules as it grows."

07 · INTERVIEW PRACTICE

Core questions: database and workflows

9. Explain the data model.

"User stores authentication and role data. Student stores student/contact/class data and optionally points to one User. Subject stores a unique catalogue entry and maximum score. Mark links a Student and Subject and stores one score per exam and academic year."

10. Why separate User and Student?

"A login identity and an academic profile are different concepts. Keeping them separate allows role and authentication data to remain independent of student/contact fields, and the nullable unique relationship allows a profile to be unlinked from a login if the workflow ever requires it."

11. Why is the mark key composite?

"A student should have one score per subject per exam per academic year. The composite unique constraint encodes that domain rule in the database; the route uses the same fields to find and update the existing score."

12. How does student creation work?

"The admin form collects profile and login details. The route checks username and admission number uniqueness, requires the student password, hashes it, adds the user, flushes to obtain its ID, adds the linked student profile, commits, and redirects."

13. Why use flush before commit there?

"The user row needs to be inserted so its generated primary key is available to assign to Student.user_id, while still keeping both objects in the same transaction before the final commit."

14. How are marks created or updated?

"The route validates selected IDs and score, loads the student and subject, compares the score to that subject's max_marks, then searches the composite key. It updates marks_obtained if found or adds a new Mark otherwise."

15. What happens if marks exceed the subject maximum?

"The route flashes a validation error and does not commit a mark. The form's general score cap is not enough by itself because each subject can have its own maximum."

16. Explain the result calculation.

"For the selected student, exam, and year, the helper joins subjects, sums achieved marks and each subject's possible marks, then computes the percentage. It applies a separate grade threshold function. With no maximum it returns 0% safely; with no marks it returns a dash grade."

17. Is the average a simple mean of subject percentages?

"No. It divides total earned marks by total possible marks, so subjects with different maximums are weighted by their available points."

18. Why use foreign keys and ORM relationships?

"Foreign keys express references between rows, and ORM relationships make it practical to navigate from a student to marks or from a mark to its subject. They improve consistency and readability; production configuration should also explicitly verify database FK enforcement and deletion policy."

08 · INTERVIEW PRACTICE

Core questions: access and security

19. How do authentication and authorization differ?

“Authentication establishes who is signed in; authorization determines what that identity may do. Flask-Login handles session identity, while role decorators and ownership checks enforce permission.”

20. How does a role-protected route work?

“The roles_required decorator applies login_required and then checks current_user.role against the permitted roles. It aborts with 403 if the role is not allowed. Templates may hide actions, but server checks are the security boundary.”

21. How do you prevent a student from viewing another student's profile?

“The profile route compares the requested Student.user_id with current_user.id when the current role is student and aborts with 403 on a mismatch. The results route builds a one-student list from the logged-in user's profile.”

22. Are passwords encrypted?

“They are hashed, not reversibly encrypted. The app uses Werkzeug's password hash generation and verification helpers. Hashing is appropriate because the application only needs to verify a submitted password, not recover it.”

23. What does CSRF protection do?

“It helps prevent a third-party site from causing a logged-in user's browser to submit an unauthorized state-changing form. Flask-WTF provides the token through hidden_tag; it does not replace authorization or input validation.”

24. Is hiding admin buttons enough to protect admin actions?

“No. A user can call a URL directly. The server route must check authorization. In this app, admin-only actions are guarded at the route in addition to being conditionally rendered in templates.”

25. What security issue would you fix first?

“Before deployment I would fail closed if SECRET_KEY is missing, remove or disable demo accounts, run behind HTTPS with a production server, and add tests for student data isolation. I would also review the GET logout route and require POST with CSRF protection for logout.”

26. What does the active flag do?

“Login checks that the account is active before accepting credentials. Admins can toggle other accounts. A stronger production implementation should also decide whether disabling an account invalidates already-existing sessions.”

27. What should be said about student privacy?

“Student data is sensitive. Use fictional data in the demo. Real deployment needs school approval, least privilege, secure secrets, HTTPS, backups, retention policies, auditability, and a privacy/security review.”

09 · INTERVIEW PRACTICE

Core questions: UI, testing, and deployment

28. What is WTForms doing?

"It defines form fields and server-side validators. Flask-WTF integrates forms with Flask and CSRF. The templates display validation errors beside fields so the user can correct input."

29. How is the interface made responsive?

"Bootstrap supplies responsive components and grid utilities; the custom stylesheet adjusts the sidebar, grids, forms, and spacing at narrower breakpoints. The app is still server-rendered HTML."

30. How is user text rendered safely?

"Jinja autoescapes ordinary interpolated template values by default in HTML templates, which mitigates many HTML injection cases. I would still avoid marking user input safe and test security boundaries explicitly."

31. What tests would you write first?

"Flask test-client tests for login success/failure, inactive accounts, role access, student A being blocked from student B's profile/results, student creation validation, marks above a subject maximum, duplicate mark update behavior, and grade boundary calculations."

32. Does this repository include an automated test suite?

"No automated test suite is included in the current starter project. I would state that honestly and add focused route and helper tests before treating it as production-ready."

33. What happens at startup?

"An application context is pushed, db.create_all creates missing tables, then seed_demo_data adds demo users/student/subjects/marks if the user table is empty. This is convenient for a demo, but it is not a schema migration strategy."

34. Why is db.create_all not enough for evolving production data?

"It can create missing tables, but it does not provide versioned, reviewed migrations for changing existing columns or constraints. I would add Flask-Migrate/Alembic and apply migrations explicitly during deployment."

35. How would you deploy it?

"I would configure a production WSGI server, HTTPS reverse proxy, environment-managed strong secrets, a maintained database with backups and migrations, logging/monitoring, and no debug mode or demo credentials. I would test restore and rollback procedures."

36. Why SQLite, and when would you change it?

"SQLite is convenient and has no separate server to configure for a local demo. For multi-user production, operational availability, concurrent writes, and scaling needs may justify PostgreSQL or another managed relational database."

10 · INTERVIEW PRACTICE

Tradeoffs, edge cases, and improvements

37. What is one limitation of the current structure?

"Most application logic lives in one app.py file. That makes a starter easy to follow, but as it grows I would split configuration, models, forms, routes/blueprints, and business services, and add an application factory for testability."

38. What could become slow as the dataset grows?

"The result register loops through students, fetches each student's exam/year combinations, then calls the result helper, which queries marks again. That can create an N+1-style query pattern. I would profile it and replace it with aggregate/join queries, eager loading, and pagination."

39. What input-modeling choices would you revisit?

"Date of birth and class are strings, and exams/years are hard-coded choices. I would use a date column and normalized/configurable academic-year, class, section, and exam records with explicit constraints."

40. How would you handle simultaneous mark submissions?

"The composite unique constraint protects the invariant, but two concurrent requests could both find no row and race to insert. In production I would handle the resulting integrity conflict or use a database-specific atomic upsert, while keeping the constraint."

41. How would you prevent accidental data loss?

"I would define retention and deletion rules, use tested backups and restoration, add audit history, and consider soft deletion for student/user records. Current ORM cascade can remove a student's marks when its record is deleted, so policy needs to be explicit."

42. What is a potential user-experience issue in current form choices?

"The marks form populates choices from existing students and subjects, but does not add a special empty choice. If either catalogue is empty, an admin/teacher cannot make a valid mark entry until records exist; I would show a helpful empty state and disable/redirect the form."

43. What would you do about demo credentials?

"Remove them from production setup, require explicit demo seeding only in development, generate non-public initial credentials or force a password change, and never rely on a known default account on an exposed service."

44. How would you improve observability?

"Use structured application logs, request/error correlation, security-relevant audit events for marks and account changes, operational metrics, and alerts. Logs must avoid exposing passwords or unnecessary student PII."

45. How would you scale the codebase?

"First keep the relational model but move to a managed database if needed. Add pagination and optimized queries, application factory/config separation, blueprints/services, tests and migrations. Only introduce a separate frontend/API if actual product requirements call for it."

11 · QUICK REFERENCE

Key facts to remember

Topic	Current implementation
Framework	Flask 3.x dependency range; server-rendered Jinja.
Database	SQLite by default; override with DATABASE_URL.
Authentication	Flask-Login; Werkzeug password hash helpers.
Forms	Flask-WTF/WTForms; validation and CSRF token via hidden_tag().
Roles	admin, teacher, student; server-side route checks.
Mark identity	student_id + subject_id + exam_name + academic_year.
Grade bands	A+ ≥90; A ≥80; B+ ≥70; B ≥60; C ≥50; D ≥40; F <40.
Startup	db.create_all() and demo seed when no users exist.
Configuration	SECRET_KEY, DATABASE_URL, DEMO_ADMIN_PASSWORD, FLASK_DEBUG.
Tests	No automated test suite currently included.
Production readiness	Demo only; needs secret/deployment hardening, migrations, tests, backups, audit/privacy controls.

Demo walkthrough (3–5 minutes)

1. Sign in as admin in a local demo and point out the dashboard counts and quick actions.
2. Open Students; explain search/class filtering and the student profile.
3. Explain that adding a student also creates a linked login with a hashed password.
4. Open Subjects and explain maximum marks.
5. Open Enter marks; show student, subject, exam/year choices and subject-maximum rule.
6. Open Results; explain filters, totals, percentage, grade, and student-only visibility.
7. Optionally sign in as student to demonstrate role-specific dashboard and data scoping.
8. Close with honest demo/production caveats and next steps.

Strong closing answer

> "The project demonstrates the full path from authenticated user input to relational persistence and role-scoped results. My next priority would be automated tests for authorization and calculations, followed by explicit migrations and deployment hardening. I would not use the current demo credentials or development server for real student data."

Preparation tip: Practice explaining one workflow end to end and be ready to point to the model, route, form, template, and database rule involved. Interviewers value clear reasoning and honest tradeoffs more than claiming the starter is production-ready.